Made by Alejandro Blanco

Similarity-Based Phishing Domain Detection



Index

1. Introduction 3
2. Objectives 3
3. Data Source 3
4. Data Extraction 4
5. Preprocessing 5
6. Similarity Analysis 6
7. Clustering 7
8. Base Domain Identification 8
9. Detection of New Domains 9
10. Results 9
11. Evaluation and Limitations 10
12. Conclusion 10
13. References 10

1. Introduction

This project explores a similarity-based method for identifying potentially malicious domains as a practical exploration of how simple similarity metrics can already provide useful signals for phishing detection.

2. Objectives

- Extract domain names from public Certificate Transparency records.
- Group lexically similar domain names.
- Identify base (original) domains within each group.
- Detect newly registered domains that may represent phishing attempts.
- Provide an explainable and academically suitable detection pipeline.

3. Data Source

Domain names were obtained from crt.sh, a public search engine for Certificate Transparency logs. The following query was used:

https://crt.sh/?q=%.com&output=json

-This source was selected because it provides a broad and unbiased view of real domain registrations observed on the internet.

4. Data Extraction

Only valid domain-like strings were extracted from the name_value field of certificate records. Subdomains were removed to retain registered domain names only. This filtering step is necessary to remove certificate metadata and other irrelevant values that could interfere with later similarity calculations.

```python
import json
import re

input_file = "../data/crtsh_raw.json"
output_file = "../data/domains_raw.txt"

domains = set()

with open(input_file, "r", encoding="utf-8") as f:
    data = json.load(f)

for entry in data:
    if "name_value" not in entry:
        continue

    values = entry["name_value"].split("\n")

    for v in values:
        v = v.strip().lower()
        if re.match(r"^[a-z0-9.-]+\.[a-z]{2,}$", v):
            parts = v.split(".")
            if len(parts) >= 2:
                domain = parts[-2] + "." + parts[-1]
                domains.add(domain)

with open(output_file, "w") as f:
    for d in sorted(domains):
        f.write(d + "\n")

print(f"{len(domains)} domains extracted")
```

```
vaca@Granja:~/Desktop/phishing-domain-detection/src$ python3 extract_domains.py
2976 domains extracted
vaca@Granja:~/Desktop/phishing-domain-detection/src$ head ../data/domains_raw.txt
1-2-sports.com
12ha.com
17photo.com
1800securitycamera.com
1stop-wallhangings.com
220mm.com
247livebet.com
2525r.com
2alljobs.com
2gig.com
```

5. Preprocessing

Domain names were normalised by converting them to lowercase and removing top-level domains. This ensures that similarity calculations focus on lexical structure. Without this step, superficial differences such as capitalisation or top-level domains could distort similarity scores.

```python
import csv

input_file = "../data/domains_raw.txt"
output_file = "../data/domains_clean.csv"

domain_counts = {}

with open(input_file, "r", encoding="utf-8") as f:
    for line in f:
        domain = line.strip().lower()
        if not domain:
            continue
        parts = domain.split(".")
        if len(parts) < 2:
            continue

        name = parts[0]

        if name not in domain_counts:
            domain_counts[name] = {
                "domain": domain,
                "count": 0
            }

        domain_counts[name]["count"] += 1

with open(output_file, "w", newline="", encoding="utf-8") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["name", "domain", "count"])

    for name, data in sorted(domain_counts.items()):
        writer.writerow([name, data["domain"], data["count"]])

print(f"{len(domain_counts)} unique domain names written")
```

```
vaca@Granja:~/Desktop/phishing-domain-detection/src$ python3 preprocess.py
2966 unique domain names written
vaca@Granja:~/Desktop/phishing-domain-detection/src$ head ../data/domains_clean.csv
name,domain,count
1-2-sports,1-2-sports.com,1
12ha,12ha.com,1
17photo,17photo.com,1
1800securitycamera,1800securitycamera.com,1
1stop-wallhangings,1stop-wallhangings.com,1
220mm,220mm.com,1
247livebet,247livebet.com,1
2525r,2525r.com,1
2alljobs,2alljobs.com,1
```

6. Similarity Analysis

Levenshtein similarity was used to compare domain names, as it provides a simple and well-understood way to measure small typographical variations commonly used in phishing domains. This metric is particularly suitable for detecting common phishing patterns such as character omission, substitution, or duplication.

```python
import csv
from itertools import combinations
import Levenshtein

input_file = "../data/domains_clean.csv"
output_file = "../data/similar_pairs.csv"
SIMILARITY_THRESHOLD = 0.8

names = []
with open(input_file, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        names.append(row["name"])

print(f"{len(names)} names loaded")

similar_pairs = []

for name1, name2 in combinations(names, 2):
    score = Levenshtein.ratio(name1, name2)
    if score >= SIMILARITY_THRESHOLD:
        similar_pairs.append((name1, name2, round(score, 2)))

print(f"{len(similar_pairs)} similar pairs found")

with open(output_file, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name1", "name2", "similarity"])
    for pair in similar_pairs:
        writer.writerow(pair)

print(f"Similar pairs written to {output_file}")
```

```
vaca@Granja:~/Desktop/phishing-domain-detection/src$ python3 similarity.py
2966 names loaded
77 similar pairs found
Similar pairs written to ../data/similar_pairs.csv
vaca@Granja:~/Desktop/phishing-domain-detection/src$ head ../data/similar_pairs.csv
name1,name2,similarity
60minuteonlinecash,60minuteonlineloans,0.86
absbook,casebook,0.8
ad-digital,aycdigital,0.8
ad-digital,pazdigital,0.8
addaxtactical,shaddoxtactical,0.86
adultsensations,adultsensationscash,0.88
agadomarketing,ramarketing,0.8
asmautoparts,astonparts,0.82
athlonline,hlcconline,0.8
```

7. Clustering

Similar domains were represented as a graph and clustered using connected components. Each cluster represents a group of related domain names. Graph-based clustering allows related domains to be grouped without predefining the number of clusters.

```python
import csv
from collections import defaultdict, deque

input_file = "../data/similar_pairs.csv"
output_file = "../data/clusters.txt"

graph = defaultdict(set)

with open(input_file, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        name1 = row["name1"]
        name2 = row["name2"]
        graph[name1].add(name2)
        graph[name2].add(name1)

print(f"{len(graph)} nodes in graph")

visited = set()
clusters = []

def bfs(start):
    cluster = set()
    queue = deque([start])
    visited.add(start)
    while queue:
        node = queue.popleft()
        cluster.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return cluster

for node in graph:
    if node not in visited:
        cluster = bfs(node)
        clusters.append(cluster)

print(f"{len(clusters)} clusters found")

with open(output_file, "w", encoding="utf-8") as f:
    for i, cluster in enumerate(clusters, 1):
        f.write(f"Cluster {i}: {', '.join(sorted(cluster))}\n")

print(f"Clusters written to {output_file}")
```

```
vaca@Granja:~/Desktop/phishing-domain-detection/src$ python3 clustering.py
104 nodes in graph
46 clusters found
Clusters written to ../data/clusters.txt
vaca@Granja:~/Desktop/phishing-domain-detection/src$ head ../data/clusters.txt
Cluster 1: 60minuteonlinecash, 60minuteonlineloans
Cluster 2: absbook, casebook
Cluster 3: ad-digital, aycdigital, pazdigital
Cluster 4: addaxtactical, shaddoxtactical
Cluster 5: adultsensations, adultsensationscash
Cluster 6: agadomarketing, ramarketing
Cluster 7: asmautoparts, astonparts
Cluster 8: athlonline, hlcconline
Cluster 9: avatarmastercourse, avatarmastersresource
Cluster 10: avatarprocourse, theavatarcourse
```

8. Base Domain Identification

For each cluster, a base domain was selected as the domain that is, on average, the most similar to the other domains in the cluster, measured using Levenshtein distance. This domain is treated as the representative (base) domain and is later used for comparison with newly observed domains.

```python
import csv

clusters_file = "../data/clusters.txt"
domains_clean_file = "../data/domains_clean.csv"
output_file = "../data/base_domains.csv"

domain_freq = {}
with open(domains_clean_file, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        name = row["name"]
        domain = row["domain"]
        count = int(row["count"])
        domain_freq[name] = count

clusters = []
with open(clusters_file, "r", encoding="utf-8") as f:
    for line in f:
        if line.strip():
            parts = line.strip().split(": ", 1)
            if len(parts) == 2:
                members = [m.strip() for m in parts[1].split(",")]
                clusters.append(members)

output_data = []
for idx, members in enumerate(clusters, 1):
    members_sorted = sorted(members, key=lambda x: (len(x), -domain_freq.get(x, 0)))
    base_domain = members_sorted[0]
    output_data.append((idx, base_domain, "|".join(members)))

with open(output_file, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["cluster_id", "base_domain", "members"])
    for row in output_data:
        writer.writerow(row)

print(f"{len(output_data)} base domains written to {output_file}")
```

```
vaca@Granja:~/Desktop/phishing-domain-detection/src$ python3 base_domain.py
46 base domains written to ../data/base_domains.csv
vaca@Granja:~/Desktop/phishing-domain-detection/src$ head ../data/base_domains.csv
cluster_id,base_domain,members
1,60minuteonlinecash,60minuteonlinecash|60minuteonlineloans
2,absbook,absbook|casebook
3,ad-digital,ad-digital|aycdigital|pazdigital
4,addaxtactical,addaxtactical|shaddoxtactical
5,adultsensations,adultsensations|adultsensationscash
6,ramarketing,agadomarketing|ramarketing
7,astonparts,asmautoparts|astonparts
8,athlonline,athlonline|hlcconline
9,avatarmastercourse,avatarmastercourse|avatarmastersresource
```

9. Detection of New Domains

New domains are compared against the base domain list. Domains exceeding the similarity threshold are flagged as potential phishing attempts. This step simulates how the system could be used to analyse newly registered domains in a real monitoring scenario.

```python
import csv
import Levenshtein

BASE_DOMAINS_FILE = "../data/base_domains.csv"
NEW_DOMAINS_FILE = "../data/new_domains.txt"
OUTPUT_FILE = "../data/suspect_domains.csv"
LEV_THRESHOLD = 0.75

base_domains = []
with open(BASE_DOMAINS_FILE, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        base_domains.append(row["base_domain"].strip().lower())

print(f"{len(base_domains)} base domains loaded")

new_domains = []
with open(NEW_DOMAINS_FILE, "r", encoding="utf-8") as f:
    for line in f:
        d = line.strip().lower()
        if not d:
            continue

        parts = d.split(".")
        if len(parts) < 2:
            continue

        name = ".".join(parts[:-1])
        new_domains.append((d, name))

print(f"{len(new_domains)} new domains loaded")

suspects = []

for full_domain, name in new_domains:
    for base in base_domains:
        lev_score = Levenshtein.ratio(name, base)

        if lev_score >= LEV_THRESHOLD or base in name:
            suspects.append(
                (full_domain, base, round(lev_score, 2))
            )

with open(OUTPUT_FILE, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["new_domain", "matched_base", "similarity"])
    for row in suspects:
        writer.writerow(row)

print(f"{len(suspects)} suspect domains written to {OUTPUT_FILE}")
```

```
vaca@Granja:~/Desktop/phishing-domain-detection/src$ python3 detect_new.py
46 base domains loaded
4 new domains loaded
4 suspect domains written to ../data/suspect_domains.csv
vaca@Granja:~/Desktop/phishing-domain-detection/src$ cat ../data/suspect_domains.csv
new_domain,matched_base,similarity
absbok.com,absbook,0.92
addax-tactical.net,addaxtactical,0.96
ramarketing-secure.org,ramarketing,0.76
ad-digital-login.com,ad-digital,0.77
```

10. Results

The system successfully detected multiple suspicious domains closely resembling known base domains. These results demonstrate the effectiveness of similarity-based detection. Several detected examples clearly show typical typosquatting behaviour, such as missing letters or the addition of misleading keywords.

11. Evaluation and Limitations

Despite this, the approach effectively demonstrates the core concept of similarity-based phishing domain detection in an explainable manner. It does not confirm actual phishing, only lexical similarity, so false positives are possible.

12. Conclusion

This project presents a complete and explainable pipeline for phishing domain detection using public data and string similarity metrics. The modular design of the implementation also makes it easy to extend or adapt to other datasets.

13. References

Certificate Transparency Logs : https://crt.sh
Levenshtein (1966)